[승코의 차근차근 Flutter] 09, Database(Sqflite)(2)

✅ Flutter `sqflite` 완전 마스터: 문법 + 사소한 디테일까지

1️⃣ SQLite의 특성부터 이해하자

🔶 SQLite는 어떤 DB인가?

특징	설명
파일 기반	앱 내부에 `.db` 파일로 저장됨 (`todo.db` , `user.db` 등)
무서버	서버 없이 자체적으로 데이터 관리
경량	수 메가바이트 수준의 가벼운 DB
트랜잭션 지원	BEGIN → COMMIT → ROLLBACK 가능
Boolean 타입 없음	`0(false)` 과 `1(true)` 로 대체

특징	설명
테이블 수정 어려움	`ALTER TABLE`이 제한적 (보통 DROP 후 재생성)

2️⃣ 데이터베이스 초기화 과정 완전 설명

🔷 파일 위치와 권한

```
String dbPath = await getDatabasesPath(); // Android: /data/data/패키지명/dat
abases
String path = join(dbPath, 'todo.db');    // 최종 경로 조합
```

> join()은 슬래시 누락 등 방지용. 경로 오류를 예방.

🔷 DB 열기

```
Database db = await openDatabase(
  path,
  version: 1,
  onCreate: _onCreate,
);
```

`openDatabase`에서 꼭 아는 3가지

콜백	의미
`onCreate`	DB 최초 생성될 때 호출됨
`onUpgrade`	버전 업그레이드 시 호출됨
`onOpen`	매번 열릴 때마다 실행됨

> 🧠 onUpgrade()는 테이블 변경이 필요할 때 사용됨. 테이블 구조 바꾸면 꼭 버전도 올려야 함.

3️⃣ SQL 문법 다시 보기 (Flutter에서 써야 하니까 정확히 알아야 함)

```
CREATE TABLE todos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  isDone INTEGER NOT NULL
);
```

키워드	의미
`INTEGER PRIMARY KEY`	SQLite에서는 자동 증가 키 (자동으로 `AUTOINCREMENT` 효과)
`TEXT`	문자열 타입
`NOT NULL`	반드시 값이 있어야 함
`BOOLEAN` 없음	대신 `INTEGER` 0/1 사용
`DATETIME` 없음	`TEXT` 나 `INTEGER` (timestamp)로 사용

> 실무에서는 createdAt TEXT처럼 날짜를 문자열 또는 timestamp(int)로 직접 저장해야 함.

4️⃣ CRUD 완전 정리 + 문법 팁

✅ INSERT

```
int id = await db.insert(
  'todos',
  {'title': '공부하기', 'isDone': 0},
  conflictAlgorithm: ConflictAlgorithm.replace,
);
```

옵션	설명
`conflictAlgorithm`	중복 발생 시 처리 방법 (replace, ignore, abort 등)

옵션	설명
`nullColumnHack`	모든 값이 null일 때 지정 열에 null 넣기 위한 꼼수 (거의 사용 X)

> conflict 해결 전략은 실무에서 중요! 기본은 abort(실패).

✅ SELECT

```
List<Map<String, dynamic>> result = await db.query(
  'todos',
  where: 'isDone = ?',
  whereArgs: [0],
  orderBy: 'id DESC',
);
```

자주 쓰는 옵션

옵션	설명
`columns`	가져올 컬럼 지정
`where` + `whereArgs`	조건문 처리 (`?` + 값 분리로 SQL 인젝션 방지)
`orderBy`	정렬 조건 (`createdAt DESC` 등)
`limit` / `offset`	페이징 처리에 사용

✅ UPDATE

```
await db.update(
  'todos',
  {'isDone': 1},
  where: 'id = ?',
  whereArgs: [3],
);
```

> 반환값은 변경된 row 개수 (int)

✅ DELETE

```
await db.delete(
  'todos',
  where: 'id = ?',
  whereArgs: [3],
);
```

> 삭제된 행 수 반환됨

5️⃣ rawQuery, rawInsert, rawUpdate, rawDelete 사용법

```
await db.rawInsert('INSERT INTO todos (title, isDone) VALUES (?, ?)', ['자바공부', 0]);
await db.rawUpdate('UPDATE todos SET isDone = ? WHERE id = ?', [1, 2]);
await db.rawDelete('DELETE FROM todos WHERE id = ?', [2]);
List<Map<String, dynamic>> rows = await db.rawQuery('SELECT * FROM todos WHERE isDone = ?', [0]);
```

> 직접 SQL을 쓸 수 있어 복잡한 쿼리 처리에 좋지만, 실수할 가능성 높음
> → 초보자는 query()사용 추천

6️⃣ Model ↔ Map 변환 설명

📌 toMap()

```
Map<String, dynamic> toMap() => {
  'id': id,
  'title': title,
```

```
  'isDone': isDone ? 1 : 0,
};
```

> DB에 저장할 때는 Dart의 bool을 int(0/1)로 변환해야 함.

📌 fromMap()

```
factory Todo.fromMap(Map<String, dynamic> map) => Todo(
  id: map['id'],
  title: map['title'],
  isDone: map['isDone'] == 1,
);
```

> 조회할 때는 다시 int → bool로 복원 필요

7️⃣ 싱글톤 DatabaseHelper 클래스 디테일 설명

```
class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._internal();
  static Database? _database;

  DatabaseHelper._internal();

  factory DatabaseHelper() => instance;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
```

```
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, 'todo.db');
    return await openDatabase(path, version: 1, onCreate: _onCreate);
  }

  // ...
}
```

❗ 왜 싱글톤으로 만들까?

- 앱 실행 중 하나의 DB 인스턴스만 유지하기 위해
- 여러 위젯에서 동시에 접근할 때 충돌 방지

8️⃣ 비동기 처리 주의

```
await db.insert(...); // 필수
```

> await를 빼먹으면 실행은 되지만, 완료되기 전에 다음 코드로 넘어가 예상치 못한 버그 발생 가능

9️⃣ 기타 자주 묻는 디테일

📌 DB 초기화하고 싶을 때

```
await deleteDatabase(join(await getDatabasesPath(), 'todo.db'));
```

📌 테이블 삭제

```
await db.execute('DROP TABLE IF EXISTS todos');
```

📌 테이블 컬럼 변경?

SQLite는 테이블 컬럼 변경이 어렵기 때문에 보통:

1. 백업 테이블 생성
2. 기존 데이터 복사
3. 원 테이블 삭제 후 재생성
4. 복사된 데이터 다시 삽입

🔟 최종 팁 모음 (진짜 사소하지만 실무에선 중요)

항목	팁
bool	SQLite엔 없음 → `INTEGER (0/1)` 사용
DateTime	`TEXT` (ISO 형식) 또는 `INTEGER` (timestamp)로 저장
SQL 문법 오류	작은 따옴표 `'`, 세미콜론 `;`, 띄어쓰기 주의
데이터 정렬	`ORDER BY` 는 항상 쿼리에서 직접 지정해줘야 함
WHERE 조건 실수	문자열 비교는 반드시 `'value'` 로 감싸기 (`title = '공부'`)
개발 중 테이블 변경	꼭 버전 올려서 `onUpgrade` 구현해야 함
상태 관리	sqflite는 UI와 자동 연동되지 않음 → `Provider` 나 `setState` 로 직접 갱신 필요

✅ 결론

> sqflite는 문법적으로 자유롭고 강력하지만, 개발자가 정확하게 SQL과 상태 흐름을 이해하고 있어야 안전하게 사용할 수 있습니다.